



SISTEMA AQUAORBIT

Gerenciamento Inteligente de Purificação Hídrica no Semiárido

DOCUMENTAÇÃO DE PROJETO (10 PONTOS)

Componente Curricular: Domain Driven Design Using Java

Curso: Análise e Desenvolvimento de Sistemas (ADS)

Disciplina Base: Domain Driven Design Using Java

Global Solution: Ano Letivo 2026 / 1º Semestre

Responsável / Integrante: Guilherme Gomes Nogueira

Sumário

1. Descritivo Completo do Projeto	3
1.1 Explicação Detalhada	3
1.2 Justificativas e Objetivos Estratégicos	3
2. Modelagem do Sistema e Diagrama de Classes	4
2.1 Arquitetura OO e Organização de Pacotes	4
3. Ilustração e Descrição dos Métodos Funcionais	5
3.1 Funcionalidades Principais de Negócio	5
4. Implementação Técnica em Java (Código Fonte)	6

1. Descritivo Completo do Projeto

1.1 Explicação Detalhada

O projeto AquaOrbit aborda diretamente a crise e escassez hídrica crônica enfrentada por comunidades geograficamente isoladas no semiárido brasileiro. Inspirando-se nos sistemas avançados de filtragem e reciclagem de fluidos da Estação Espacial Internacional (ISS), a iniciativa propõe a implantação e a gestão inteligente de módulos portáteis descentralizados para a purificação da água.

A infraestrutura computacional apresentada consiste em um ecossistema modular desenvolvido em Java, pautado pelas melhores práticas do Domain Driven Design (DDD). O software centraliza o controle operacional e laboratorial de todas as estações de tratamento instaladas no campo. O principal objetivo do sistema é assegurar um fornecimento sustentável, contínuo e seguro de água potável através do monitoramento automatizado e em tempo real de parâmetros de qualidade de amostras hídricas, eliminando riscos à saúde pública e quantificando os indicadores de impacto social gerados pelo projeto.

1.2 Justificativas e Objetivos Estratégicos

A implementação do sistema AquaOrbit justifica-se plenamente pela criticidade associada ao controle operacional tradicional. Processos analíticos manuais estão intrinsecamente sujeitos a falhas de digitação e atrasos operacionais que colocam populações vulneráveis em risco imediato por contaminação. Através da informatização e validação das entradas em nível de objeto (garantindo dados inteiros de população e valores decimais exatos para ensaios físico-químicos), o software blindar a operação e automatiza a tomada de decisão em campo.

Os objetivos estratégicos bem definidos do escopo compreendem:

- Efetuar o cadastramento robusto e a indexação sistemática de todas as comunidades e regiões mapeadas pelo projeto.
- Garantir o controle patrimonial rigoroso do inventário de hardware de purificação, discriminando tecnologicamente os purificadores por radiação UV Compacta e os baseados em Destilação Solar.
- Implementar mecanismos de associação (atributos de referência) capazes de vincular equipamentos a locais de consumo de forma dinâmica e escalável.
- Instituir um motor de regras capaz de confrontar variáveis analíticas de pH, turbidez e cloro residual com portarias ministeriais vigentes de potabilidade.
- Fornecer auditorias automáticas que suspendam e alterem o estado operacional de equipamentos caso qualquer amostra apresente desconformidade.
- Mensurar dinamicamente o impacto social total da iniciativa por meio do cálculo computacional centralizado das vidas assistidas pelo projeto.

2. Modelagem do Sistema e Diagrama de Classes

2.1 Arquitetura OO e Organização de Pacotes

Em total conformidade com os requisitos estipulados de Domain Driven Design e Programação Orientada a Objetos, a aplicação foi projetada utilizando um modelo rico que contém um mínimo absoluto de 6 classes interdependentes, encapsuladas e estruturadas em pacotes lógicos definidos:

Detalhamento das Relações Estruturais e Arquitetura:

- **Organização em Pacotes:** As classes que compõem o modelo de domínio residem adequadamente no pacote estrutural `aquaorbit.entidades`, enquanto o ponto centralizado de inicialização e interface via Scanner situa-se no pacote executável `aquaorbit.main`.
- **Mecanismo de Herança:** A classe abstrata `Equipamentos` unifica as propriedades fundamentais do inventário. As especializações técnicas `PurificadorUV` (com propriedades de potência da lâmpada em watts) e `PurificadorDestilacao` (com capacidade volumétrica nominal em litros por hora) estendem a classe base de forma legítima, demonstrando amplo domínio de herança e reaproveitamento de código.
- **Atributo de Referência:** A classe `Equipamentos` possui uma associação estrutural direta com a classe `Comunidade`, estabelecendo a rastreabilidade geográfica do dispositivo em campo. Da mesma maneira, a classe de negócio `Monitoramento` referencia o objeto abstrato `Equipamentos` correspondente de onde a amostra hídrica foi extraída.
- **Encapsulamento e Integridade:** Todas as variáveis de instância são restritas por visibilidade estritamente privada (`private`). O acesso e modificação ocorrem unicamente através de métodos acessores públicos (getters e setters), contendo validações dedicadas, como o impedimento de populações com valores numéricos negativos na classe `Comunidade`.

3. Ilustração e Descrição dos Métodos Funcionais

3.1 Funcionalidades Principais de Negócio

Excluindo por completo métodos utilitários, construtores padrão e propriedades básicas de acesso (getters/setters), o projeto implementa de forma explícita quatro métodos complexos de lógica de negócio para a manipulação dos dados inseridos:

Método 1: `avaliarQualidadeAgua()` [Classe: Monitoramento]

Descrição e Regras de Negócio: Executa um processamento lógico e condicional booleano cruzando três variáveis analíticas decimais independentes. A amostra é validada positivamente (retornando `true`) se, e somente se, o pH mantiver-se entre 6.0 e 9.5, a turbidez for inferior ou igual a 5.0 NTU e o cloro residual situar-se entre 0.2 e 2.0 mg/L. Caso qualquer um dos índices viole as margens de tolerância, o método retorna `false`.

Método 2: `obterResultadoFormatado()` [Classe: Monitoramento]

Descrição e Regras de Negócio: Consome o estado lógico processado pelo avaliador de qualidade hídrica (Método 1) e executa um desvio condicional estruturado para gerar uma saída de dados em formato textual descritivo amigável, retornando a mensagem "APROVADA para consumo." ou o alerta formal de erro "REPROVADA - Risco à saúde pública."

Método 3: `calcularTotalVidasImpactadas(Comunidade c)` [Classe: Main]

Descrição e Regras de Negócio: Método estático que atua como um gerador de relatórios de impacto social. Ele isola e extrai com segurança os indicadores populacionais validados de dentro do objeto `Comunidade` passado por parâmetro de referência, exibindo de forma clara a quantidade total de pessoas beneficiadas pela respectiva unidade operacional.

Método 4: `emitirAlertaSeguranca(Monitoramento m)` [Classe: Main]

Descrição e Regras de Negócio: Intercepta de forma reativa possíveis anomalias operacionais nos dados monitorados. Se o método interno de potabilidade do objeto `Monitoramento` retornar falso, o motor dispara uma saída emergencial priorizada contendo o código identificador único do hardware afetado (obtido via associação na árvore de objetos) para triagem técnica imediata.

4. Implementação Técnica em Java (Código Fonte)

Abaixo encontra-se transcrita a arquitetura física e sintática integral do código fonte que compõe a solução computacional desenvolvida:

Pacote: aquaorbit.entidades | Classe: Comunidade.java

```
package aquaorbit.entidades;

public class Comunidade {
    private String nome;
    private String localizacao;
    private int populacaoAtendida;

    public Comunidade() {}

    public Comunidade(String nome, String localizacao, int populacaoAtendida) {
        this.nome = nome;
        this.localizacao = localizacao;
        setPopulacaoAtendida(populacaoAtendida);
    }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getLocalizacao() { return localizacao; }
    public void setLocalizacao(String localizacao) { this.localizacao = localizacao; }

    public int getPopulacaoAtendida() { return populacaoAtendida; }

    // Validação estrita: garante população inteira positiva
    public void setPopulacaoAtendida(int populacaoAtendida) {
        if (populacaoAtendida >= 0) {
            this.populacaoAtendida = populacaoAtendida;
        } else {
            this.populacaoAtendida = 0;
        }
    }

    @Override
    public String toString() {
        return "Comunidade: " + nome + " | Localização: " + localizacao + " | "
        + "População: " + populacaoAtendida;
    }
}
```

Pacote: aquaorbit.entidades | Classe Abstrata: Equipamentos.java

```
package aquaorbit.entidades;

public abstract class Equipamentos {
    private String codigoIdentificacao;
    private String status; // Ativo, Manutenção, Inativo
    private Comunidade comunidadeVinculada; // Atributo de Referência

    public Equipamentos() {}

    public Equipamentos(String codigoIdentificacao, String status, Comunidade comunidadeVinculada) {
        this.codigoIdentificacao = codigoIdentificacao;
        this.status = status;
        this.comunidadeVinculada = comunidadeVinculada;
    }

    public String getCodigoIdentificacao() { return codigoIdentificacao; }
    public void setCodigoIdentificacao(String codigoIdentificacao)
    { this.codigoIdentificacao = codigoIdentificacao; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public Comunidade getComunidadeVinculada() { return comunidadeVinculada; }
    public void setComunidadeVinculada(Comunidade comunidadeVinculada)
    { this.comunidadeVinculada = comunidadeVinculada; }

    @Override
    public String toString() {
        return "Código: " + codigoIdentificacao + " | Status: " + status + " | " +
        (comunidadeVinculada != null ? comunidadeVinculada.getNome() : "Sem comunidade");
    }
}
```

Pacote: aquaorbit.entidades | Classe: PurificadorUV.java

```
package aquaorbit.entidades;

public class PurificadorUV extends Equipamentos {
    private double potencialLampadaWatts;

    public PurificadorUV() {}
}
```

```
    public PurificadorUV(String codigoIdentificacao, String status, Comunidade
comunidadeVinculada, double potenciaLampadaWatts) {
        super(codigoIdentificacao, status, comunidadeVinculada);
        this.potenciaLampadaWatts = potenciaLampadaWatts;
    }

    public double getPotenciaLampadaWatts() { return potenciaLampadaWatts; }
    public void setPotenciaLampadaWatts(double potenciaLampadaWatts)
{ this.potenciaLampadaWatts = potenciaLampadaWatts; }

    @Override
    public String toString() {
        return super.toString() + " | Tipo: UV Compacto (Potência: " +
potenciaLampadaWatts + "W)";
    }
}
```

Pacote: aquaorbit.entidades | Classe: PurificadorDestilacao.java

```
package aquaorbit.entidades;

public class PurificadorDestilacao extends Equipamentos {
    private double capacidadeLitrosHora;

    public PurificadorDestilacao() {}

    public PurificadorDestilacao(String codigoIdentificacao, String status, Comunidade comunidadeVinculada, double capacidadeLitrosHora) {
        super(codigoIdentificacao, status, comunidadeVinculada);
        this.capacidadeLitrosHora = capacidadeLitrosHora;
    }

    public double getCapacidadeLitrosHora() { return capacidadeLitrosHora; }
    public void setCapacidadeLitrosHora(double capacidadeLitrosHora)
    { this.capacidadeLitrosHora = capacidadeLitrosHora; }

    @Override
    public String toString() {
        return super.toString() + " | Tipo: Destilação Solar (Capacidade: " +
        capacidadeLitrosHora + " L/h)";
    }
}
```

Pacote: aquaorbit.entidades | Classe: Monitoramento.java

```
package aquaorbit.entidades;

public class Monitoramento {
    private double ph;
    private double turbidez;
    private double cloroResidual;
    private Equipamentos equipamentoMonitorado;

    public Monitoramento() {}

    public Monitoramento(double ph, double turbidez, double cloroResidual, Equipamentos equipamentoMonitorado) {
        this.ph = ph;
        this.turbidez = turbidez;
        this.cloroResidual = cloroResidual;
        this.equipamentoMonitorado = equipamentoMonitorado;
    }
}
```

```

// --- MÉTODOS FUNCIONAIS DE NEGÓCIO ---

public boolean avaliarQualidadeAgua() {
    return (this.ph >= 6.0 && this.ph <= 9.5) &&
        (this.turbidez <= 5.0) &&
        (this.cloroResidual >= 0.2 && this.cloroResidual <= 2.0);
}

public String obterResultadoFormatado() {
    if (avaliarQualidadeAgua()) {
        return "APROVADA para consumo.";
    } else {
        return "REPROVADA - Risco à saúde pública.";
    }
}

public double getPh() { return ph; }
public void setPh(double ph) { this.ph = ph; }

public double getTurbidez() { return turbidez; }
public void setTurbidez(double turbidez) { this.turbidez = turbidez; }

public double getCloroResidual() { return cloroResidual; }
public void setCloroResidual(double cloroResidual) { this.cloroResidual =
cloroResidual; }

public Equipamentos getEquipamentoMonitorado() { return equipamentoMonitorado; }
public void setEquipamentoMonitorado(Equipamentos equipamentoMonitorado)
{ this.equipamentoMonitorado = equipamentoMonitorado; }

@Override
public String toString() {
    return "Análise -> pH: " + ph + ", Turbidez: " + turbidez + " NTU, Cloro: " +
cloroResidual + " mg/L";
}
}

```

Pacote: aquaorbit.main | Classe Executável: Main.java

```
package aquaorbit.main;

import aquaorbit.entidades.*;
import java.util.Scanner;

public class Main {

    public static void main(String[] academico) {
        Scanner entrada = new Scanner(System.in);

        System.out.println("--- SISTEMA AQUAORBIT - STARTUP OPERACIONAL ---");

        System.out.print("Digite o nome da comunidade a ser atendida: ");
        String nomeCom = entrada.nextLine();

        System.out.print("Digite a localização/estado: ");
        String locCom = entrada.nextLine();

        System.out.print("Digite a população estimada: ");
        int popCom = entrada.nextInt();
        entrada.nextLine();

        Comunidade comunidade = new Comunidade(nomeCom, locCom, popCom);

        PurificadorUV purificadorUv = new PurificadorUV("EQ-UV01", "Ativo",
        comunidade, 15.5);
        PurificadorDestilacao purificadorDes = new PurificadorDestilacao("EQ-DS02",
        "Manutenção", comunidade, 45.0);

        System.out.println("\n--- REGISTRO DE TESTE FÍSICO-QUÍMICO ---");
        System.out.print("Informe o pH coletado na amostra: ");
        double ph = entrada.nextDouble();

        System.out.print("Informe o nível de turbidez (NTU): ");
        double turbidez = entrada.nextDouble();

        System.out.print("Informe o cloro residual (mg/L): ");
        double cloro = entrada.nextDouble();

        Monitoramento analise = new Monitoramento(ph, turbidez, cloro, purificadorUv);

        System.out.println("\n=====");
        System.out.println("                RELATÓRIO CONSOLIDADO EM TEMPO REAL");
        System.out.println("=====");
```

```

        System.out.println(comunidade);
        System.out.println("\nInventário de Equipamentos:");
        System.out.println(" -> " + purificadorUv);
        System.out.println(" -> " + purificadorDes);

        System.out.println("\nResultado do Monitoramento:");
        System.out.println(" -> " + analise);
        System.out.println(" -> Status da Amostra: " +
analise.obterResultadoFormatado());

        calcularTotalVidasImpactadas(comunidade);
        emitirAlertaSeguranca(analise);

        System.out.println("=====");
        entrada.close();
    }

    public static void calcularTotalVidasImpactadas(Comunidade c) {
        System.out.println("\n[MÉTODO 3 - Métrica Social]");
        System.out.println(" >> Sucesso! Total de " + c.getPopulacaoAtendida() + "
vidas positivamente impactadas pela iniciativa.");
    }

    public static void emitirAlertaSeguranca(Monitoramento m) {
        System.out.println("\n[MÉTODO 4 - Auditoria de Risco]");
        if (!m.avaliarQualidadeAgua()) {
            System.out.println(" >> ALERTA URGENTE: Desative o equipamento "
                + m.getEquipamentoMonitorado().getCodigoIdentificacao()
                + " imediatamente para triagem técnica!");
        } else {
            System.out.println(" >> Operação Estável. Parâmetros dentro das metas de
qualidade.");
        }
    }
}

```